

KEPLER-7 APLICADO A 12 PATRONES DE DISEÑO

IÑAKI FARIAS, AGUSTÍN QUIROZ, BRUNO ROCA

UNIVERSIDAD ABIERTA INTERAMERICANA

3A MAÑANA - CENTRO 091

PEREYRA JORGE AGUSTÍN, SILVESTRO LUCAS AGUSTÍN

ÍNDICE

INFORME TÉCNICO – Kepler-7 Juego 3
1. Descripción del juego y del género 3
2. Arquitectura 4
3. Doce patrones 5
BUILDER 5
PROTOTYPE 6
BRIDGE 7
FACADE 8
FLYWEIGHT 9
PROXY 10
CHAIN OF RESPONSABILITY 11
COMMAND 12
VISITOR 13
STATE 14
STRATEGY 15
TEMPLATE METHOD 16
4. Dificultades encontradas 17
5. Diferencias entre patrones creacionales, estructurales y de comportamiento .. 18
6. Aspectos a modifica o mejorar en versiones posteriores 18

INFORME TÉCNICO – Kepler-7 Juego

1. Descripción del juego y del género

Kepler-7 es un juego de estrategia táctica por turnos para escritorio, hecho en C# con Windows Forms sobre .NET 8, sin motores ni paquetes de terceros. Una plaga alienígena tomó la Estación Orbital Kepler-7. El jugador es el Capitán Kael Ortiz y comanda un escuadrón de cuatro (Capitán, Soldado, Médica y Francotiradora) contra seis aliens, uno por sector. Cada victoria restaura un subsistema de la estación, y cada subsistema es uno de los patrones que prepara la misión siguiente.

Ciclo de juego. El jugador prepara la misión en la estación (nave, clon, mensaje a la Tierra, destacamento, bóveda), inicia el turno desde el Control Central, combate por turnos, recibe un resumen con XP y logros, y vuelve a la estación con un subsistema nuevo. Se gana al vencer al Leviatán del Vacío, el jefe del sector 6, y se pierde si cae el Capitán; el sector puede reintentarse sin perder el progreso.

Progreso y gamificación. Una pantalla inicial con los integrantes, un tutorial de cinco pantallas, XP y niveles (150 XP por nivel, con rangos de Cadete a Comandante de flota), un panel de logros que marca los patrones dominados, una pantalla final con el resumen de la campaña y persistencia en un JSON dentro de %AppData%\Kepler7 con System.Text.Json. Las respuestas visuales usan Timer, cambios de color y controles propios dibujados con OnPaint; la estética es oscura, de colores apagados. Sumamos además un modo sandbox con todos los módulos desbloqueados y sin guardado, que usamos para la demostración.

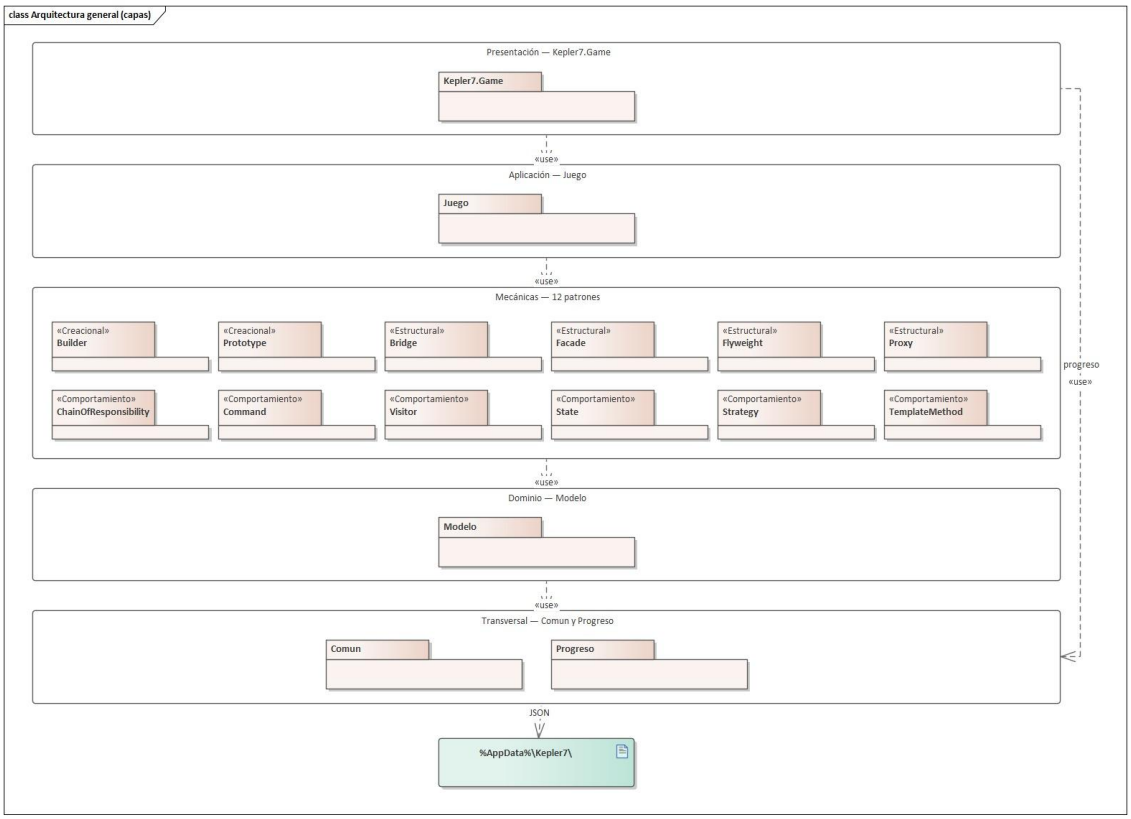
Por qué este género. Elegimos combate táctico por turnos con progresión porque cada decisión del jugador es un punto discreto donde un patrón puede intervenir y ser observado: la orden (Command), el turno que dispara (Template Method), el estado del personaje (State), el daño recibido (Chain of Responsibility) y la decisión del enemigo (Strategy). El turno también facilita separar la lógica de la pantalla. Los cinco patrones que no son de combate viven en la estación, que se reconstruye entre misiones.

2. Arquitectura

La solución tiene dos proyectos y se organiza en cinco capas, donde cada una solo usa a las de abajo. Arriba está la capa de Presentación (Kepler7.Game), que contiene únicamente los formularios y controles de Windows Forms: muestra el estado del juego y transmite las acciones del usuario. Todo lo demás está en Kepler7.Core, que no referencia a System.Windows.Forms.

Dentro del Core, la capa de Aplicación (Juego) coordina la partida: Partida es el punto de entrada de la interfaz y organiza el combate, la fachada ControlCentral y los módulos de la estación. Debajo, la capa de Mecánicas reúne los doce patrones (creacionales, estructurales y de comportamiento), cada uno asociado a una mecánica concreta del juego. Esos patrones trabajan sobre la capa de Dominio (Modelo), que define las entidades del juego, como Personaje, MiembroEscuadron y Enemigo.

En la base está la capa Transversal, con los servicios que usan todas las demás: Comun (registro de eventos y aleatoriedad) y Progreso (XP, logros y desbloqueos). El progreso se guarda en archivos JSON en %AppData%\Kepler7, y la interfaz lo carga directamente al iniciar el juego. El Core avisa a la interfaz mediante eventos y registros, por ejemplo, cuando un patrón queda dominado, así la lógica nunca depende de la pantalla.



3. Doce patrones

Cada patrón resuelve una mecánica concreta y observable del juego. Los dos creacionales y los cuatro estructurales viven en la estación; los seis de comportamiento, en el combate. Para cada uno indicamos el problema, la solución, dónde se ve y por qué lo elegimos.

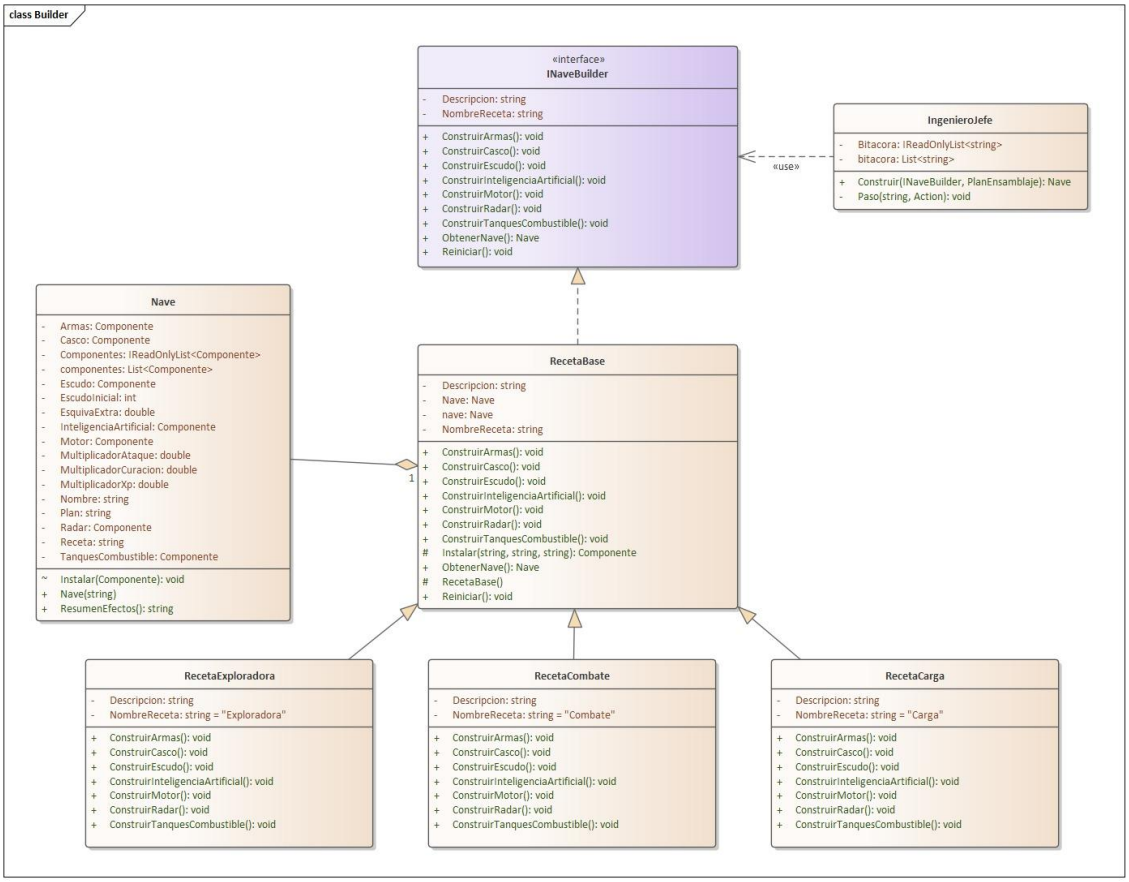
BUILDER

Problema. Una nave tiene siete componentes opcionales (casco, motor, escudo, armas, radar, IA y tanques). Un constructor con siete parámetros anulables o una clase por combinación no escalan.

Solución. INaveBuilder define un paso por componente; RecetaBase y las recetas RecetaExploradora, RecetaCombate y RecetaCarga fabrican su versión de cada pieza; IngenieroJefe (director) decide qué pasos ejecutar y en qué orden según el plan (Completo, Asalto o Reconocimiento). El producto es Nave.

¿Dónde se ve? En el Hangar la nave aparece pieza por pieza y la bitácora lista los métodos que llamó el director. Sus bonificaciones se aplican en las misiones.

¿Por qué? Separa el proceso de armado de las piezas: tres recetas por tres planes dan nueve naves sin una clase por combinación.



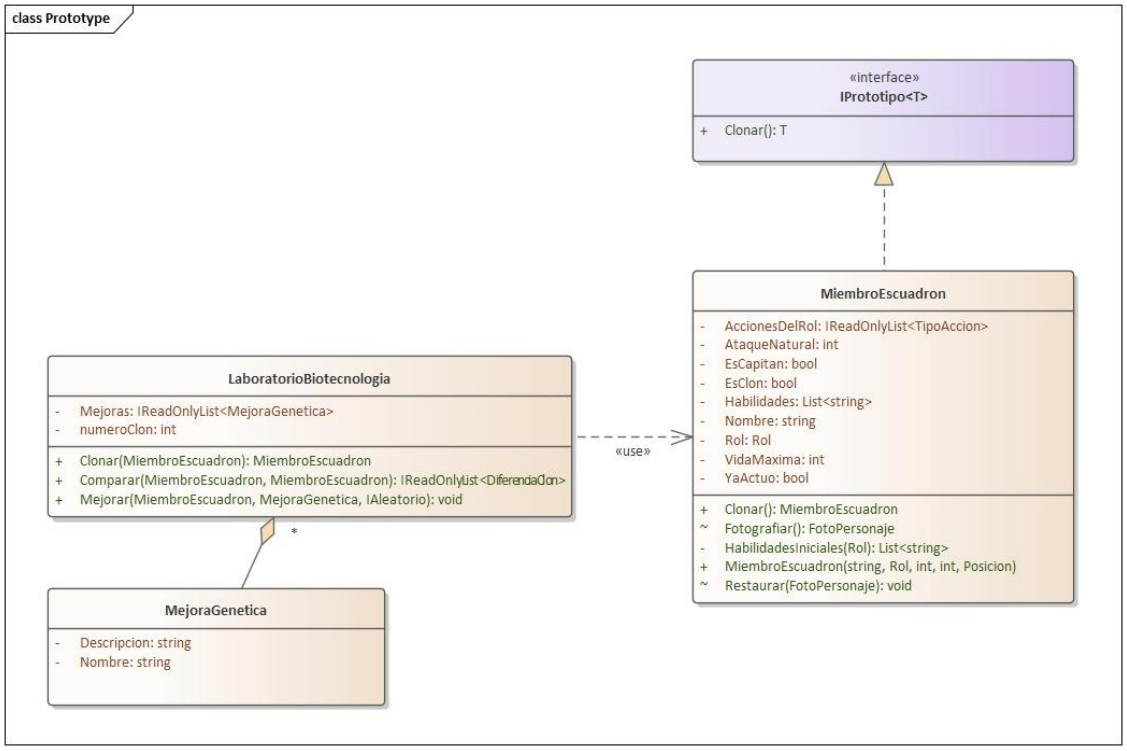
PROTOTYPE

Problema. Un tripulante tiene estado anidado (lista de habilidades, cadena de defensas). Rearmarlo desde cero obliga a repetir su configuración, y MemberwiseClone compartiría las listas.

Solución. IPrototipo<T> define Clonar(); MiembroEscuadron hace una copia profunda (nueva lista de habilidades y nueva cadena con copias de los eslabones; el eslabón Vida no se clona porque pertenece a su dueño). LaboratorioBiotecnologia clona, aplica una MejoraGenetica y arma la DiferenciaClon.

¿Dónde se ve? Una tabla compara original y clon y comprueba con ReferenceEquals que no comparten referencias. En combate, el clon releva al primer caído.

¿Por qué? Conserva toda la configuración del original y permite modificar solo lo que cambia, sin afectar al tripulante base.



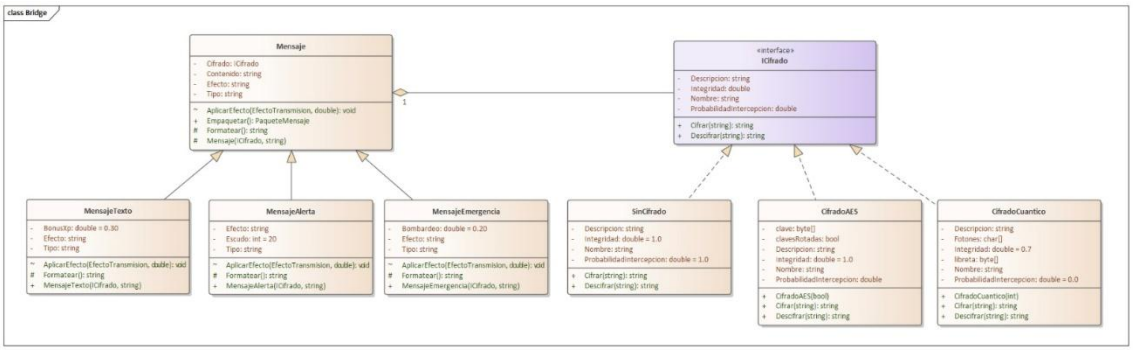
BRIDGE

Problema. Un mensaje varía en dos dimensiones independientes: su tipo (Texto, Alerta, Emergencia) y su cifrado (ninguno, AES, cuántico). Con herencia serían nueve clases y crecerían al multiplicarse.

Solución. Mensaje y sus tres refinadas forman una jerarquía; ICifrado, con SinCifrado, CifradoAES y CifradoCuantico, forma la otra. La referencia que Mensaje guarda a ICifrado es el puente. CentroComunicaciones combina ambas en ejecución.

¿Dónde se ve? La consola muestra el formato del mensaje, la salida del cifrado y el resultado. Texto da XP +30%, Alerta da escudo +20 y Emergencia baja un 20% la vida inicial del alien; sin cifrado el alien siempre intercepta, con AES el 30% y con el cuántico nunca (a 70% de potencia).

¿Por qué? Seis clases cubren nueve combinaciones, y sumar un tipo o un cifrado es sumar una clase.



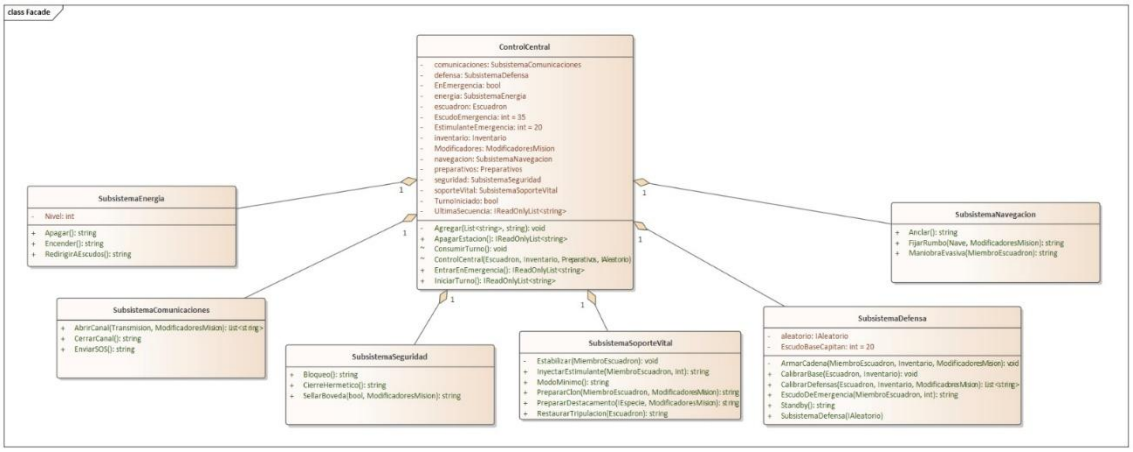
FACADE

Problema. Preparar una misión exige coordinar seis subsistemas (Energía, Soporte Vital, Navegación, Comunicaciones, Seguridad y Defensa) en el orden correcto. Si el formulario los llamara uno por uno, conocería todos.

Solución. ControlCentral ofrece IniciarTurno(), EntrarEnEmergencia() y ApagarEstacion(). Los subsistemas son internal, así que el formulario no puede invocarlos directamente.

¿Dónde se ve? Una ventana muestra la secuencia coordinada, línea por línea. IniciarTurno() convierte los preparativos de la estación en modificadores de combate, y la emergencia se activa sola cuando el Capitán baja del 30% de vida.

¿Por qué? Simplifica el uso y concentra en un punto la conexión entre la estación y el combate.



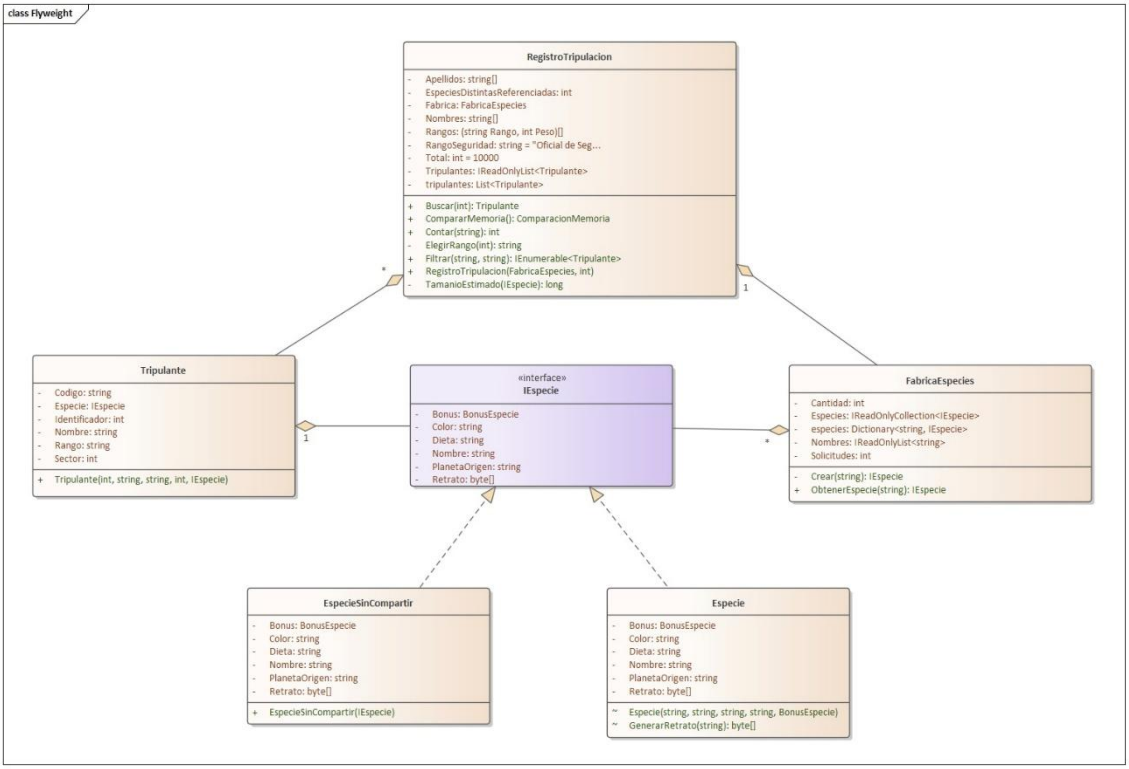
FLYWEIGHT

Problema. Hay 10.000 tripulantes y solo cinco especies. Duplicar en cada tripulante los datos de su especie desperdicia memoria.

Solución. IEspecie/Especie guardan el estado compartido (nombre, color, planeta, dieta); Tripulante guarda lo propio (nombre, identificador, rango) y una referencia a su especie. FabricaEspecies usa un Dictionary<string, IEspecie> para garantizar una sola instancia por especie.

¿Dónde se ve? Un contador muestra 10.000 pedidos a la fábrica y solo 5 objetos creados. Una comparación real contra EspecieSinCompartir pasa de unos 54 MB a unos 21 KB. Además, cada especie otorga un bonus al destacamento.

¿Por qué? Es el único patrón cuyo beneficio se mide directamente en memoria.



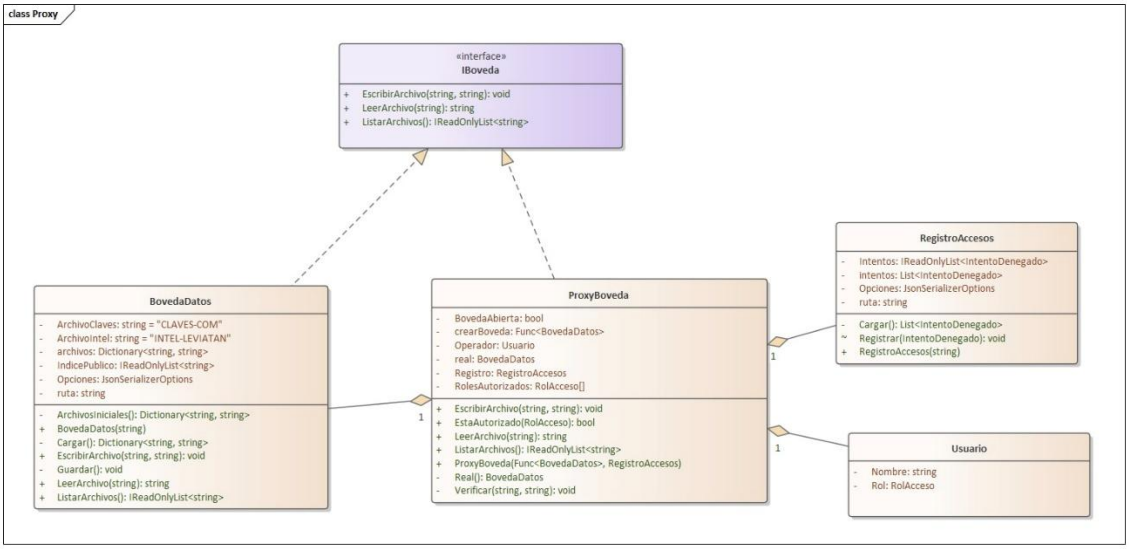
PROXY

Problema. La bóveda guarda información reservada y solo el Capitán y el Oficial de Seguridad pueden leerla o modificarla.

Solución. IBoveda define LeerArchivo y EscribirArchivo; BovedaDatos los ejecuta y ProxyBoveda verifica el rol antes de delegar. Es un proxy de protección con carga diferida: la bóveda real se crea al primer acceso autorizado. RegistroAccesos guarda los rechazos en un JSON dentro de %AppData%\Kepler7.

¿Dónde se ve? Un rechazo hace parpadear la terminal y suma una línea al registro visible. Leer el informe debilita al jefe final y rotar las claves anula la intercepción del AES.

¿Por qué? Controla el acceso sin modificar la clase real ni a sus clientes.



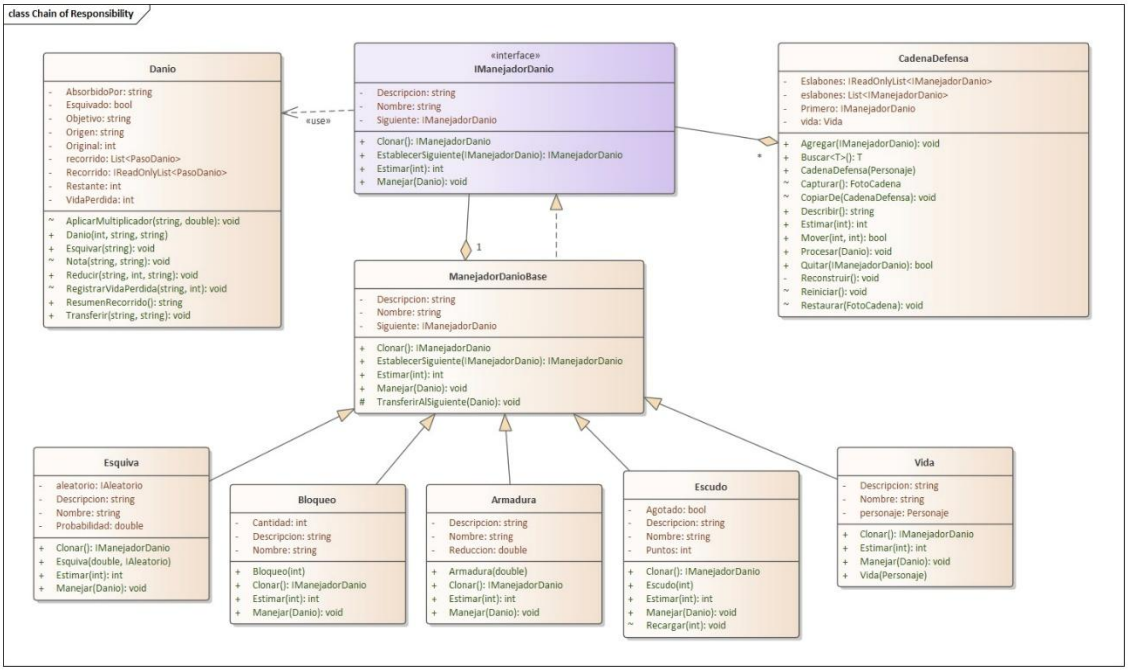
CHAIN OF RESPONSABILITY

Problema. El daño debe atravesar esquiva, bloqueo, armadura, escudo y vida, y el orden cambia el resultado. Un único método con condicionales habría que modificarlo con cada defensa nueva.

Solución. IManejadorDanio define Manejar(Danio); ManejadorDanioBase guarda el siguiente eslabón. Esquiva, Bloqueo, Armadura, Escudo y Vida absorben lo que pueden y transfieren el resto. Cada Personaje tiene su CadenaDefensa, que permite agregar, quitar y reordenar eslabones en ejecución.

¿Dónde se ve? Un panel dibuja el recorrido del último golpe. Construir agrega un escudo a la cadena de un aliado y un escudo agotado se retira solo.

¿Por qué? Cada defensa es una clase chica y la cadena se arma por personaje; una prueba automática verifica que reordenar eslabones cambia el resultado.



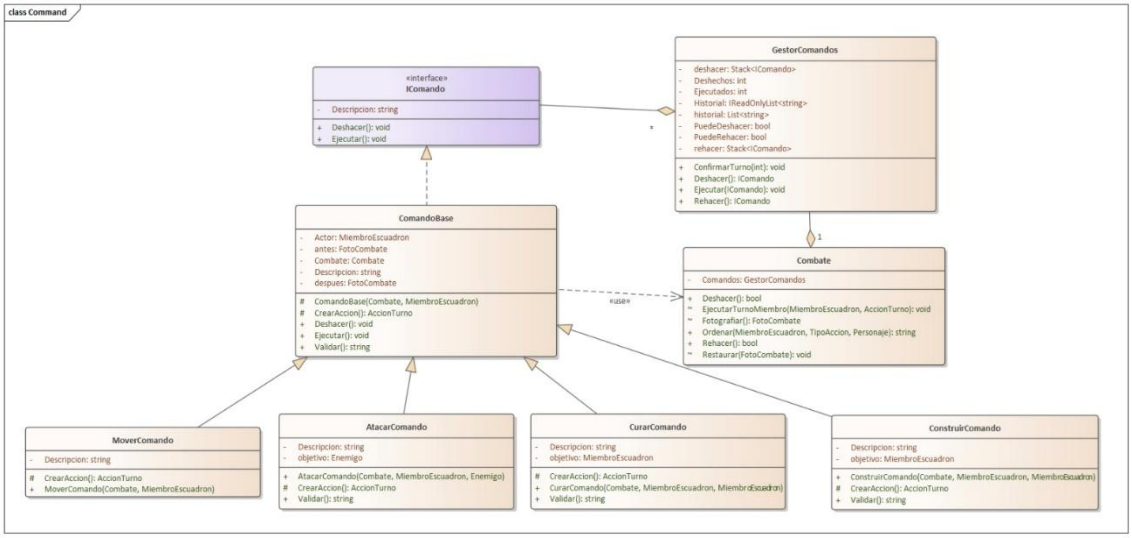
COMMAND

Problema. El jugador debe poder probar una jugada y arrepentirse antes de confirmar el turno, y necesitamos un historial de lo que hizo.

Solución. IComando define Ejecutar() y Deshacer(); MoverComando, AtacarComando, CurarComando y ConstruirComando heredan de ComandoBase. GestorComandos es el invocador, con pilas de deshacer y rehacer, y Combate es el receptor. Cada comando guarda una foto del combate antes y después de ejecutarse.

¿Dónde se ve? Botones Deshacer y Rehacer (Ctrl+Z / Ctrl+Y) y un historial en la pestaña Órdenes. Al confirmar el turno las pilas se vacían, y rehacer restaura la foto.

¿Por qué? Convierte la acción en un objeto con historial y reversión.



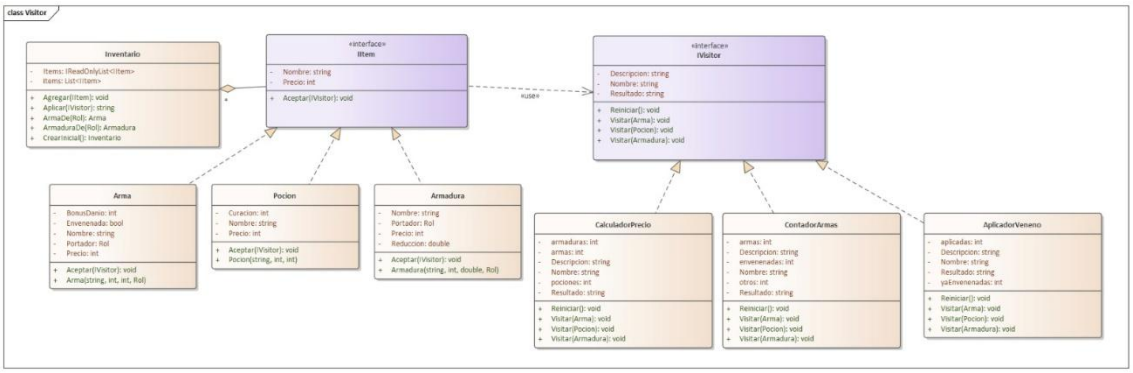
VISITOR

Problema. El inventario tiene tres tipos de elemento (Arma, Poción, Armadura) y queremos sumar operaciones sin modificar esas clases.

Solución. IItem define Aceptar(IVisitor); IVisitor tiene un Visitar por tipo. CalculadorPrecio, ContadorArmas y AplicadorVeneno son los visitantes y Inventario aplica el elegido a todos sus elementos.

¿Dónde se ve? En la pestaña Arsenal el jugador elige un visitante y ve el resultado. El Aplicador de veneno tiene efecto real: los golpes siguientes dejan envenenado al alien.

¿Por qué? La estructura es estable y las operaciones cambian; sumar una operación es sumar un visitante.



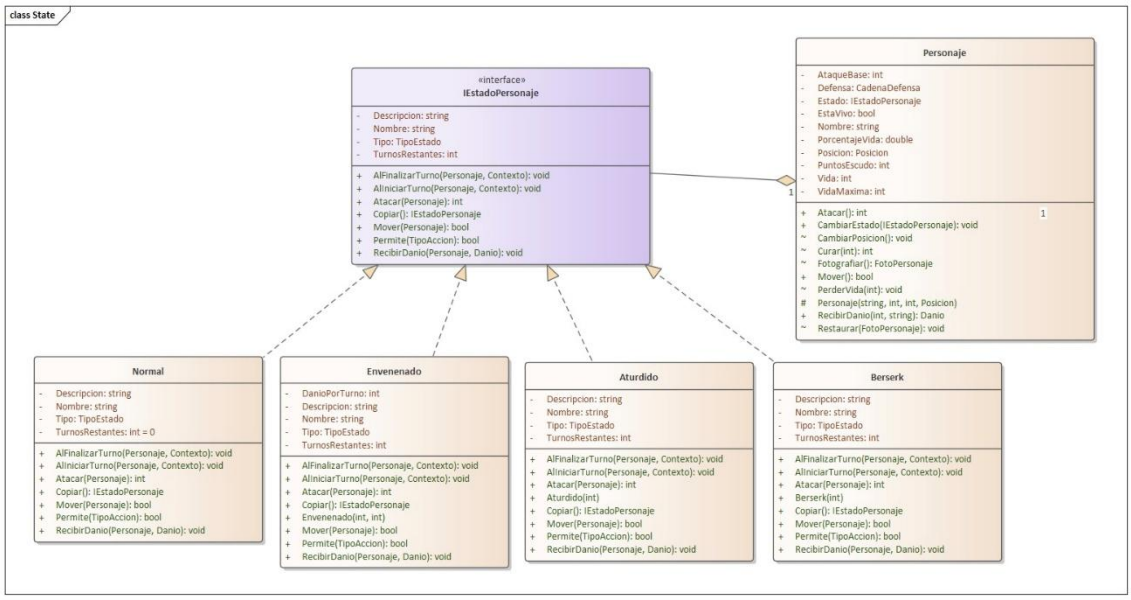
STATE

Problema. El personaje se comporta distinto según esté Normal, Envenenado, Aturdido o Berserk. Un enum con switch repetido en cada acción obligaría a tocar todo al sumar un estado.

Solución. Personaje delega Atacar, Mover y RecibirDanio en IEstadoPersonaje; cada estado es una clase. Envenenado pierde vida por turno y ataca al 80%; Aturdido no actúa y recibe 25% más de daño; Berserk ataca x1,5, recibe 25% más y solo ataca.

¿Dónde se ve? El borde de cada tarjeta cambia de color, se muestran los turnos restantes y los botones se deshabilitan según lo que el estado permite.

¿Por qué? Cada estado concentra su comportamiento y agregar uno es agregar una clase.



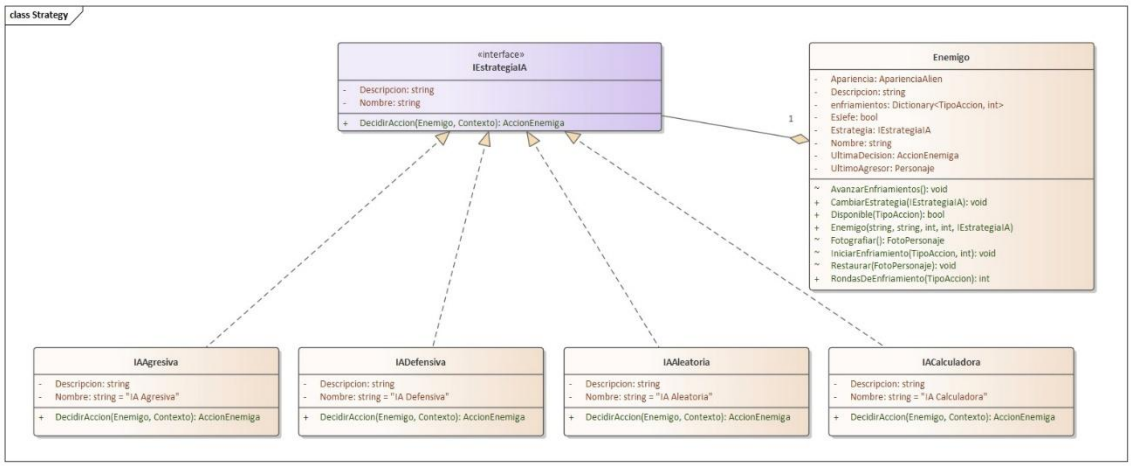
STRATEGY

Problema. Los aliens deben decidir con algoritmos distintos y el jugador debe poder cambiarlos durante la partida.

Solución. IEstrategiaIA define DecidirAccion(Enemigo, Contexto). IAAgresiva va por el Capitán, IADefensiva se cubre con poca vida, IAAleatoria elige al azar e IACalculadora estima el daño con la cadena de defensas. Enemigo.CambiarEstrategia() la reemplaza en ejecución.

¿Dónde se ve? La opción “Interferencia neural” cambia la IA de un alien y la pestaña IA registra cada decisión con la estrategia que la tomó. Si el jugador no interviene, el alien se adapta solo al bajar del 50% de vida.

¿Por qué? Los algoritmos son intercambiables y sumar una IA no modifica a Enemigo.



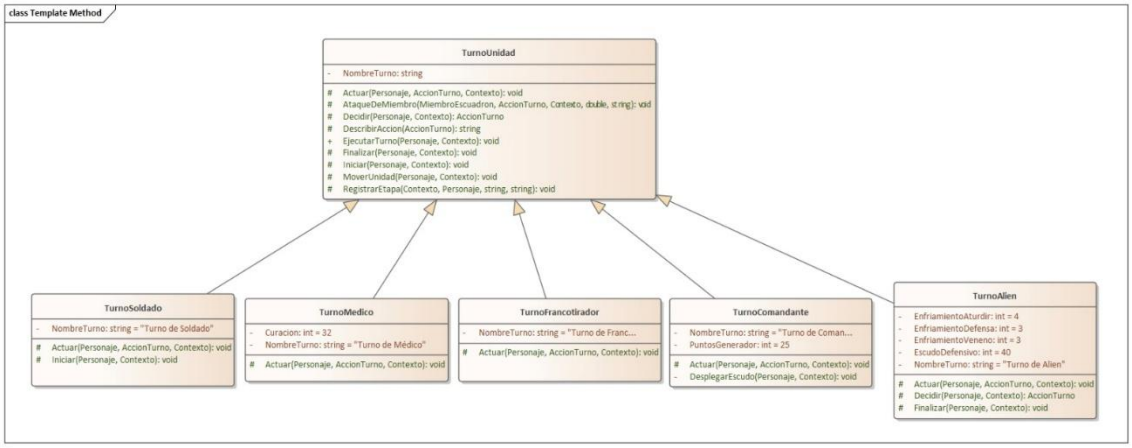
TEMPLATE METHOD

Problema. Todas las unidades siguen inicio, decisión, acción y final, pero difieren en los pasos variables. Repetir el esqueleto por rol permitiría romperlo.

Solución. TurnoUnidad.EjecutarTurno() no es virtual y fija el orden. Actuar es abstracto y Iniciar, Decidir y Finalizar son virtuales con comportamiento por defecto. Las subclases son TurnoSoldado, TurnoMedico, TurnoFrancotirador, TurnoComandante y TurnoAlien.

¿Dónde se ve? La pestaña Turnos registra cada etapa de cada unidad. El veneno se aplica siempre al inicio y un aturdido nunca actúa, sin importar el rol.

¿Por qué? Garantiza el orden común y deja variar solo lo que corresponde a cada rol.



4. Dificultades encontradas

Las dificultades que encontramos al momento de desarrollar el juego son:

- Deshacer en el patrón Command. Revertir una orden no es “deshacer un ataque”: cambian la vida, los estados, las cadenas de defensa y la formación, y la orden ejecuta además un turno completo. Lo resolvimos sacando una foto del combate antes y después de cada comando. Rehacer restaura la segunda foto en lugar de volver a ejecutar, para que el jugador no pueda repetir tiradas aleatoriamente hasta que salgan bien.
- Que ningún patrón sea decorativo. La consigna exige que cada patrón intervenga en una mecánica concreta. Facade, Flyweight y Proxy, que son estructurales y no tienen un efecto obvio en un combate, necesitaron un puente; que fueron los preparativos que dejan los cinco patrones de la estación que se convierten en modificadores de la misión a través de ControlCentral.
- Copia profunda con estado anidado en el patrón Prototype. Un tripulante lleva una lista de habilidades, una cadena de defensas y un estado con contadores propios. Hubo que copiar cada uno de forma independiente; el eslabón Vida no se clona porque pertenece a su dueño, y los estados tienen un método Copiar() para no compartir contadores entre fotos.
- Dejar el proyecto Core sin Windows Forms. Toda la comunicación hacia la pantalla pasa por eventos y registros (daño, estados, refuerzos, logros) que la interfaz consume. Exigió pensar qué necesita ver la interfaz sin que el Core sepa que existe.
- Limitaciones de Windows Forms. Los controles estándar son claros y no animan por sí solos. Pintamos de oscuro los controles estándar en una clase Tema, dibujamos las tarjetas del campo con OnPaint y un Timer, y activamos doble búfer para evitar el parpadeo. Para no bloquear el hilo de la interfaz usamos async/await: Task.Delay para las pausas del turno enemigo y Task.Run para la comparación de memoria de Flyweight.

5. Diferencias entre patrones creacionales, estructurales y de comportamiento

Los patrones creacionales como Builder y Prototype responden a cómo se debe crear un objeto. En el juego producen algo que el jugador lleva a la misión, una nave armada por pasos y un clon copiado de un tripulante.

Los patrones estructurales como Bridge, Facade, Flyweight y Proxy no agregan comportamiento nuevo, sino que cambian cómo se organizan y se conectan las clases (separar en dos jerarquías, esconder los subsistemas, compartir datos entre 10.000 objetos, interponer un control de acceso). Todos están en la estación.

Los patrones de comportamiento como Chain of Responsability, Command, Visitor, State, Template Method y Strategy reparten responsabilidades mientras el juego corre. Concretamente se usa para saber quién absorbe el daño, cómo se deshace una orden, qué hace cada estado, en qué orden transcurre un turno y quién decide la acción del enemigo. Por eso los seis patrones están implementados en la etapa de combate.

En el código se distinguen por el momento en que actúan: los creacionales en la preparación, los estructurales en la organización de la estación y los de comportamiento en cada turno. Además, no son independientes, por ejemplo, Command dispara Template Method, que consulta a State, cuyo daño entra a la cadena de Chain of Reponsability; Strategy decide dentro de la plantilla y usa la cadena para estimar; y Visitor deja el veneno que activa un estado. Cada patrón se relaciona con otro.

6. Aspectos a modificar o mejorar en versiones posteriores

Ampliaríamos la campaña con más sectores y enemigos, nuevos estados y más recetas de nave, lo que en nuestro diseño equivale a agregar clases. Incorporaríamos feedback sonoro con System.Media, que hoy no tenemos y un editor de niveles. Por último, simplificaríamos la interfaz de combate para pantallas chicas, porque su ventana es relativamente grande.